SAP Report

Design and Implementation of a Microservices-Based Application

Rosario Zaccone

June 2026

Version: v0.1.8
Last Modified: 2026-07-05
GitHub Repository: https://github.com/rosario-zaccone/rione

Contents

1 Application Description

Rione is a social network centred on small local communities. Its purpose is to support interaction among people who live in the same neighbourhood, limiting the social space to nearby residents rather than creating a broad and generic online network. The platform is based on the idea that digital communication can become more useful and meaningful when it is connected to the places in which people live their everyday lives.

The core principle of the project is neighbourhood-based interaction. Users can see and interact only with other users who belong to the same neighbourhood. For example, in a city such as Bologna, neighbourhoods may include Navile, Saragozza, San Vitale, Savena, and others. A user who belongs to Navile should interact only with other users from Navile, while users from different neighbourhoods remain outside that local social space. This constraint gives the platform a clear community boundary and distinguishes it from traditional social networks, where interactions are often global, anonymous, or weakly connected to real-life contexts.

The motivation behind Rione is to create a more local and purposeful form of online participation. By focusing on neighbourhood communities, the platform aims to strengthen relationships among nearby residents, encourage participation in local community life, and make online exchanges more relevant to practical daily needs. A local social network can help people share information, ask for help, discuss neighbourhood issues, organize small events, and become more aware of what happens around them.

Another important motivation is trust. Interactions among people who share the same local environment can be more concrete than interactions in large-scale social networks. The neighbourhood boundary encourages users to perceive other participants not as distant strangers, but as members of the same local community. This can increase the perceived value of contributions and support more responsible communication.

The platform also introduces deliberate constraints to promote more thoughtful participation. For instance, posts and comments may require a minimum length, encouraging users to write meaningful contributions instead of very short or superficial messages. These constraints are intended to increase attention, reduce low-effort interactions, and improve the quality of discussions. In this sense, Rione is designed not only as a communication tool, but also as a way to encourage more careful, relevant, and community-oriented digital participation.

2 Technological Choices

This section describes the main technologies and development tools used in the project. Architectural and design decisions are discussed in later sections.

2.1 Technologies Used

The project is implemented as a Maven multi-module application. The backend uses Java and Spring Boot, while local execution and infrastructure are provided through Docker Compose.

- **Java 21**: main programming language for backend services.
- **Spring Boot 4.1.0**: base framework for the backend microservices.

- **Spring Web MVC**: implementation of REST APIs exposed by the services.
- **Spring Data JPA**: persistence layer for relational data.
- **PostgreSQL**: relational database used by the services that require persistence.
- **Apache Kafka**: event broker used for asynchronous communication between social, post, and notification workflows.
- **Spring Security**: authentication and authorization through bearer tokens and service-to-service security rules.
- **Springdoc OpenAPI**: generation and publication of API documentation.
- **React and Vite**: frontend implementation and local development environment.
- **Docker and Docker Compose**: containerization and local orchestration of services, databases, Kafka, Prometheus, Grafana, and pgAdmin.
- **Prometheus and Grafana**: collection and visualization of technical and business metrics.
- **Maven**: build, dependency management, and test execution.
- **GitHub and GitHub Actions**: version control and continuous integration.

3 Testing Architecture

The testing strategy is organized around Test-Driven Development and layered verification. Business rules are first expressed in requirements documents and then mapped to executable tests. The goal is not only to check implementation details, but also to keep the code aligned with the expected domain behavior.

The project follows the idea of the testing pyramid. At the base of the pyramid there are many fast unit tests, because they are cheap to execute and give quick feedback on domain and application logic. In the middle there are integration tests, which are fewer because they involve frameworks, adapters, persistence, security, messaging, or HTTP boundaries. At the top there are higher-level acceptance tests, which are more expensive but validate business behavior from the user's point of view. Architecture tests are used as an additional technical guardrail to verify that the intended code structure is preserved.

Each main business service follows the same testing structure under `src/test`. The project separates acceptance tests, architecture tests, integration tests, and unit tests. This separation keeps each test focused on a specific kind of risk: business correctness, architectural conformance, adapter behavior, or isolated domain and application logic.

The test suite is executed with Maven. During continuous integration, GitHub Actions runs `verify` on each service and then on the full Maven reactor. This means that a change is validated both in its own module and in the context of the whole project.

3.1 Types of Tests

The project uses four main test layers.

- **Unit Tests**: unit tests verify small units of behavior in isolation. They are placed at the base of the testing pyramid because they are fast, deterministic, and numerous. In this project, they are implemented with JUnit and Mockito and focus on domain models, value objects, and application services. Examples include tests for user value objects such as biography, birth date, and password, and service tests for user, social, post, and notification application logic.

- **Integration Tests**: integration tests verify that multiple components work correctly together across technical boundaries. They occupy the middle level of the testing pyramid because they are slower than unit tests but provide stronger confidence in framework and adapter behavior. In this project, they are implemented with JUnit and Spring testing support and verify REST controllers, security configuration, JWT handling, JPA repositories, Kafka adapters, metrics configuration, and schema migration behavior.

- **Architecture Tests**: architecture tests verify structural constraints rather than business behavior. They check that the implementation respects the intended separation between domain, application ports, application services, and infrastructure adapters. In this project, they are implemented with ArchUnit and act as automated guardrails against accidental dependency violations.

- **Acceptance Tests**: acceptance tests verify the system behavior expected by the requirements. They are placed at the top of the testing pyramid because they validate broader business scenarios. In this project, they are implemented with Gherkin, Cucumber, and JUnit, and they are explicitly modeled on the documented business rules. Feature files use stable rule identifiers such as `USR-BR-001`, `SOC-BR-001`, `PST-BR-006`, and `NOT-BR-001`, so each acceptance scenario can be traced back to a specific business rule.

This combination gives the project broad coverage without relying on a single kind of test. Unit tests keep domain logic fast to verify, integration tests validate the collaboration with frameworks and adapters, architecture tests prevent accidental dependency violations, and acceptance tests protect the business requirements defined in the requirements documentation.

3.2 CI/CD Pipeline

The repository uses GitHub Actions to automatically verify the project. The current pipeline is focused on continuous integration: it checks that the services compile and that their automated tests pass before changes are considered safe to merge.

- The workflow runs on pushes to `feature/**` branches and to `main`.

- The workflow also runs on pull requests targeting `main`.

- A matrix job verifies the main backend services independently: User Service, Post Service, Social Service, and Notification Service.

- Each service job uses Java 21 and executes `./mvnw -B -pl <module> verify`.
- After the service-level jobs, a full Maven reactor verification is executed with `./mvnw -B verify`.
- Maven dependency caching is enabled through the Java setup action to reduce build time.

This pipeline supports a test-first and regression-aware workflow. The goal is to detect broken business rules, architectural violations, integration problems, or compilation errors before they reach the main branch.

4 Domain-Driven Design Approach

Reminder: remodel this section and explain the path followed during the domain analysis. First describe the initial general modelling of the application domain, then explain how the bounded contexts were identified, and finally describe how the analysis focused on each bounded context.

4.1 Strategic Design

4.2 Bounded Contexts

4.3 Ubiquitous Language

5 Requirements

The requirements documentation is stored in the following folders:

- `docs/requirements/api-gateway/`
- `docs/requirements/user/`
- `docs/requirements/social/`
- `docs/requirements/post/`
- `docs/requirements/notification/`

5.1 User Stories

User stories are short requirement descriptions written from the perspective of an actor. They describe who needs a capability, what they want to do, and why the capability has value.

The user stories are stored in:

- `docs/requirements/api-gateway/user_stories.md`
- `docs/requirements/user/user_stories.md`
- `docs/requirements/social/user_stories.md`

- `docs/requirements/post/user_stories.md`
- `docs/requirements/notification/user_stories.md`

5.2 Requirements Tables

Requirement tables translate user stories into structured system responsibilities. They assign stable identifiers to functional requirements, business rules, and non-functional requirements.

The requirement tables are stored in the Software Requirements Specification files:

- `docs/requirements/api-gateway/srs.md`
- `docs/requirements/user/srs.md`
- `docs/requirements/social/srs.md`
- `docs/requirements/post/srs.md`
- `docs/requirements/notification/srs.md`

5.3 Business Rules

Business rules are domain constraints that must remain true regardless of the technical interface used to execute an operation. They define when an operation is valid, what must be rejected, and which side effects are allowed.

The business rules are documented in the same requirements folders as the user stories and SRS files:

- `docs/requirements/api-gateway/`
- `docs/requirements/user/`
- `docs/requirements/social/`
- `docs/requirements/post/`
- `docs/requirements/notification/`

6 Application Architecture and Deployment

Reminder: merge here the content previously planned for the microservice architecture section. Do not use the C&C diagram in this chapter. Use a deployment diagram instead, showing how the frontend, gateway, backend services, databases, Kafka, and monitoring components are deployed and connected.

7 Main Microservices

Reminder: for each microservice, include a component diagram, a domain diagram, and a textual description.

Briefly introduce the main microservices of the system.

7.1 API Gateway

Briefly describe the role of the API Gateway implemented with Spring.

7.2 User Service

Briefly describe the User Service, its domain model, and its API design.

7.3 Social Service

Briefly describe the Social Service, its domain model, and its API design.

7.4 Post Service

Briefly describe the Post Service, its domain model, its API design, and the Event Sourcing pattern implemented inside it.

7.5 Notification Service

Briefly describe the Notification Service, its domain model, its API design, and how it uses Kafka as an event broker.

8 Additional Patterns and Monitoring

Briefly describe the additional technical patterns and monitoring tools used in the project.

8.1 Health API

Briefly describe the use of Spring Actuator to expose health endpoints.

8.2 Application Metrics

Briefly describe how Prometheus and Grafana are used to collect and visualize application metrics.

9 Implementation

Briefly describe the implementation phase of the project.

9.1 Use of AI During Coding

Briefly describe how AI was used during the coding phase and how its output was reviewed and adapted.

10 Conclusions

Reminder: write the conclusions by focusing on the analysis carried out during the project and on scalability considerations. Discuss how the adopted decomposition can support growth in users, neighbourhoods, services, data volume, and interactions, and mention possible limitations or future improvements related to scalability.